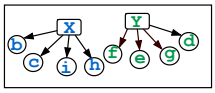


(in-weights graph)

Training data

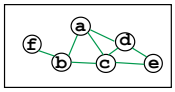
(1)"g, f, Y"
(2)"e, d, Y"
(3)"h, c, X"
(4)"b, i, X"

Generated samples

(A)"b, f, Y" (incoherent)
(B)"b, i, X" (memorized ~ (4))
(C)"e, d, Y" (memorized ~ (2))
(D)"e, d, X" (incoherent)
(E)"X, i, b" (incoherent)
(F)"b, c, X" ✓
(G)"e, g, Y" ✓
(H)"e, g, Y" (duplicated ~ (G))

Algorithmic creativity = 2 / 8

(a) Sibling Discovery



(in-weights graph)

Training data

(1) "tri: ab, bc, ca"
(2) "edge: cd, dc"
(3) "edge: ab, ba"
(4) "edge: bc, cb"
(5) "edge: fb, bf"
(6) "edge: ac, ca"
(7) "tri: da, ac, cd"
(8) "edge: ce, ec"
(9) "edge: ed, de"
(10)"edge: ad, da"

Generated samples

(A)"tri: ab, bc, ca" (memorized ~ (1))
(B)"tri: ab, bf, fa" (incoherent)
(C)"tri: bc, ca, ab" (memorized ~ (1))
(D)"tri: ca, ab, bc" (memorized ~ (1))
(E)"tri: cd, de, ea" ✓
(F)"tri: de, ea, ad" (duplicated ~ (E))
(G)"tri: dc, ce, ed" (duplicated ~ (E))
(H)"tri: bc, ce, eb" (incoherent)
(I)"tri: dc, ca, ad" (memorized ~ (7))
(J)"tri: ac, ce, ea" (incoherent)

Algorithmic creativity = 1 / 10

(b) Triangle Discovery

Figure 9. **Minimal tasks inspired by combinational creativity:** The in-weights graph represents the underlying knowledge graph used to generate the training data (not provided in-context). Based on our definition of algorithmic creativity in Eq. (1), generated samples that are incoherent or memorized, or duplicated are not counted as valid samples. Note that sequences that are permutations of each other are considered identical when computing duplicates and memorization.

C. Description of datasets

C.1. Datasets inspired by combinational creativity

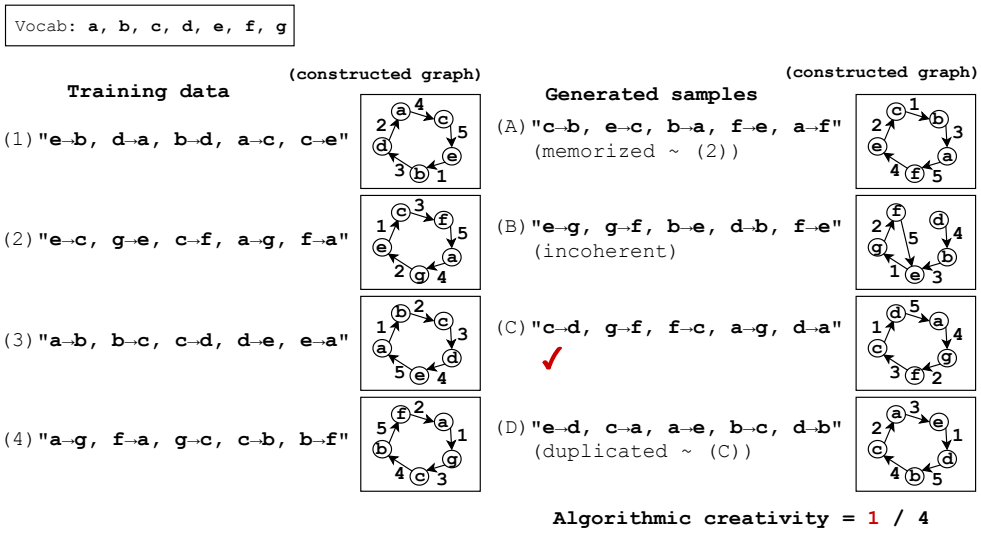
Dataset 1: `Sibling Discovery` (Fig 9(a)). This task is based off a bipartite graph $\mathcal{G}$ made of parent vertices $\mathcal{V} = \{A, B, C, \ldots\}$ each neighboring a corresponding set of children $\texttt{nbr}(A) = \{a_1, a_2, \ldots, \}$. We set the number of parent vertices $|\mathcal{V}|$ to be small and the number of children for each parent vertex $|\texttt{nbr}(A)|$ to be large. For example, $|\mathcal{V}| = 5$ and $|\texttt{nbr}(A)| = 500$. We define $\texttt{coh}(\boldsymbol{s})$ to hold on "sibling-parent" triplets of the form $\boldsymbol{s} = (\gamma, \gamma', \Gamma)$ such that $\gamma, \gamma' \in \texttt{nbr}(\Gamma)$.

Next, we ensure that the training set is large enough for the model to infer all the edges in the graph. Let $m = |\mathcal{V}|$ and $n = |\texttt{nbr}(\Gamma)|$ (for all $\Gamma \in \mathcal{V}$). This means $S = \Omega(m \cdot n)$. At the same time, to keep the task non-trivial, the training set must be small enough to not cover all the coherent sibling-parent triplets. Thus, we ensure $S = o(m \cdot n^2)$.

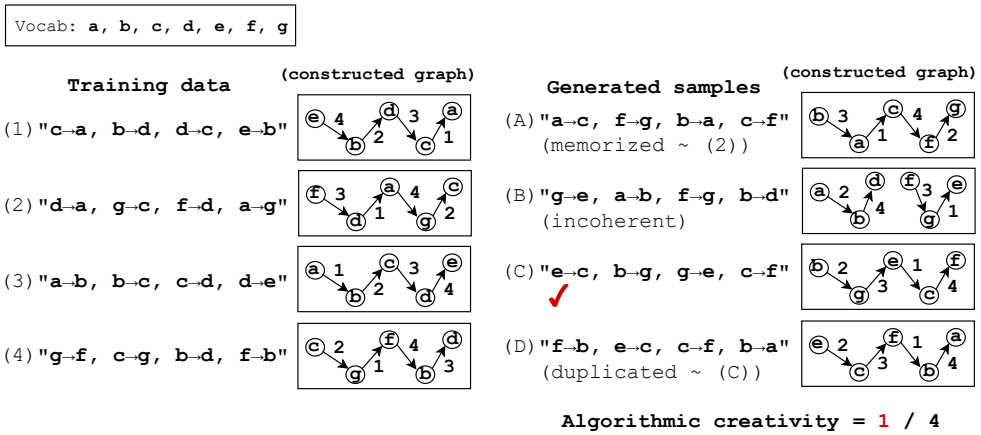
For the default version of this dataset, we set $|\mathcal{V}| = 5$ and $|\texttt{nbr}(\Gamma)| = 500$ for all $\Gamma \in \mathcal{V}$.

Dataset 2: `Triangle Discovery` (Fig 9(b)). This task is based off an undirected graph $\mathcal{G} = (V, E)$ which contains many triangles. Since a triangle is a symmetric structure, the problem remains the same even upon reordering the vertices. Thus, in this task $\texttt{coh}((v_1, v_2, v_3)) = \texttt{true}$ iff all three edges between $\{v_1, v_2, v_3\}$ belong in $\mathcal{G}$. To make this task interesting (neither too trivial nor too non-trivial) for our models to learn, we enforce several constraints on the graph. First, we try to keep the degree `deg` of each vertex to be sufficiently small. On the one hand, this is so that no vertex requires too much computation to find a triangle it is part of; on the other, we also do not want a very dense graph where most random triplets are a triangle. In addition to this degree requirement, we ensure that each vertex has a minimum number of triangles.

Thus to create a graph that is neither too trivial nor too non-trivial, we define a two-step graph generation procedure. In the first step, we iterate over the vertices, and add `deg` many edges from that vertex to other vertices in the set (where



(a) Circle Construction



(b) Line Construction

Figure 10. **Tasks inspired by exploratory creativity:** The constructed graph visualizes the graph induced by the training or generated sample. Edge indices represent the order of edge appearing in the string. Based on our definition of algorithmic creativity in Eq. (1), generated samples that are incoherent, memorized, or duplicated are not counted as valid samples. Note that sequences that correspond to the same permutations but with different participating vertices are considered identical when computing duplicates and memorization

deg is small, such as 3 or 10). To avoid creating high-degree vertices inadvertently, we only select neighbors with degree $\leq 1.2 \cdot \texttt{deg}$. This alone may not ensure a sufficient number triangles in each vertex; so we iterate over the vertices to explicitly create tri random triangles on each vertex (where tri is small, such as 6 or 10). We do this by selecting pairs of a vertex's neighbors and drawing an edge between them.

Next, we want a training dataset such that (a) the model can infer all the edges from the graph and yet (b) not all triangles appear in the dataset. This necessitates training on a dataset that consists not only of a subset of the triangles, but also of edges from the graph. Our training data consists of two parts: (1) $1/3$ are random triangles from the graph, (2) $2/3$ are random edges from the graph. In the training set, the triangle and edge samples are distinguished by a prefix "triangle:" or "edge:". During test-time, we ensure that the model is prompted with "triangle:". A triangle (u, v, w) is tokenized as "tri: $(u, v), (v, w), (w, u)$" and an edge (u, v) as "edge: $(u, v), (v, u)$". We provide both the directions of edge to potentially avoid any issues with the reversal curse (Berglund et al., 2024; Allen-Zhu & Li, 2023a).

For the default setting of the dataset, we set $|V| = 999$, $\texttt{deg} = 3$, $\texttt{tri} = 6$.

Table 1. **Hyperparameter details for `Gemma v1 (2B)` model.**

Hyperparameter	`Sibling Discovery`	`Triangle Discovery`	`Circle Construction`	`Line Construction`
Max. Learning Rate	5×10^{-4}	5×10^{-4}	5×10^{-4}	5×10^{-5}
Model Seq. Len.	32	32	2048	2048
Training steps	7500	$10k$	$15k$	$15k$
Training size	$50k$	$15k$	$10k$	$10k$
Weight given to multi-token obj.	0.5	0.5	0.75	0.75

C.2. Datasets inspired by exploratory creativity

Dataset 3: `Circle Construction` (Fig 10(a)). In this task, the generated strings must be randomized adjacency lists that can be rearranged to recover circle graphs of N vertices. The vertices come from a fixed vocabulary of M tokens. Specifically, let the generated list be $\boldsymbol{s} = (v_{i_1}, v_{i_2}), (v_{i_3}, v_{i_4}), \ldots$. We define $\texttt{coh}(\boldsymbol{s}) = \texttt{true}$ iff there exists a *resolving* permutation π such that $\pi(\boldsymbol{s}) = (v_{j_1}, v_{j_2}), (v_{j_2}, v_{j_3}), \ldots (v_{j_n}, v_{j_1})$ for distinct $j_1, j_2, \ldots j_n$. i.e., each edge leads to the next, and eventually circles back to the first vertex. In our experiments, we set M to be larger than N.

Our default experiments are reported for $N = 9, M = 15$.

Dataset 4: `Line Construction` (Fig 10(a)). This task is a minor variant of the above where the edge set E corresponds to a line graph. The details are same here except for coherence to hold, we need a resolving permutation π such that $\pi(\boldsymbol{s}) = (v_{j_1}, v_{j_2}), (v_{j_2}, v_{j_3}) \ldots, (v_{j_{n-1}}, v_{j_n})$ for distinct $j_1, j_2, \ldots j_n$. i.e., each edge leads to the next, stopping at a dead-end. We use the same set of hyperparamters as `Circle Construction`.

Our default experiments are reported for $N = 9, M = 15$.

D. Further experimental details

Details for `Gemma v1 (2B)` model. In Table 1, we provide the hyperparameter details for each of our datasets. We note some common details here. First, the batch size is 4, but each sequence is packed with multiple examples; thus the model sequence length (divided by the input length) can be treated as a multiplicative factor that determines the effective batch size. The learning rates are chosen favorable to next-token prediction (not multi-token prediction). The training steps were chosen roughly based on a point after which the model had saturated in algorithmic creativity (and exhibited decreasing creativity). We use a learning rate with linear warm up for 100 steps, followed by cosine annealing upto a factor $0.01\times$ of the maximum learning rate. To measure creativity, we sample a test dataset T of 1024 datapoints.

We represent the main tokens in our tasks with integers (ranging upwards of 0 to as many distinct integers as required). In the seed-conditioning setting, we use seeds of default length 10, using randomly sampled uppercase characters from the English alphabet. In all datasets, we space-separate the vertices in a string, and comma-separate the edges.

Details for `GPT-2 (86M)` model. In Table 2, we provide the hyperparameter details for each of our datasets. We use `GPT-2 (small)` with `86M` non-embedding parameters when we are comparing Transformers with diffusion models. We train these models with a learning rate of 10^{-4} and a batch size of 64, to convergence in terms of the algorithmic creativity. In temperature sampling, we use nucleus sampling with a parameter of 0.95, although we observed this does not affect the results qualitatively. We used top-k sampling with a parameter of 50. We also explore temperatures above 1.0 for these models since temperatures low than 1.0 only did worse. We provide sensitivity analysis of learning rate in §F.

Details for `SEDD (90M)` model. We use SEDD's "absorb" variant, which begins denoising with a fully masked sequence and iteratively refines tokens over 128 denoising steps. This variant achieves the best language modeling performance in the original paper. Same as `GPT-2 (86M)`, we train these models with a learning rate of 10^{-4} and a batch size of 64, to